Introductory Programming Techniques

North Metropolitan TAFE

ICTPRG302: Session 5

Courses:

- Certificate III in Information Technology
- Certificate IV in IT (Programming)
- Certificate IV in IT (Networking)
- Certificate IV in IT (Cybersecurity)
- Certificate IV in IT (Web)

Background created by Adrian Goold

What will I learn?

- The definition of a function
- Creating and calling functions
- Function parameters
- Function arguments
- Function return values.

To complete this week

- Weekly session exercises
- Portfolio questions
- Revision…practise, practise, practise!

Revision: loops

What can you remember? **True** or **False?**

A while loop will continue to execute when its condition is True.

Revision: loops

What can you remember? **True** or **False?**

A for loop is an infinite loop.

Revision: loops

What can you remember? **True** or **False?**

The continue statement causes the loop to terminate immediately.

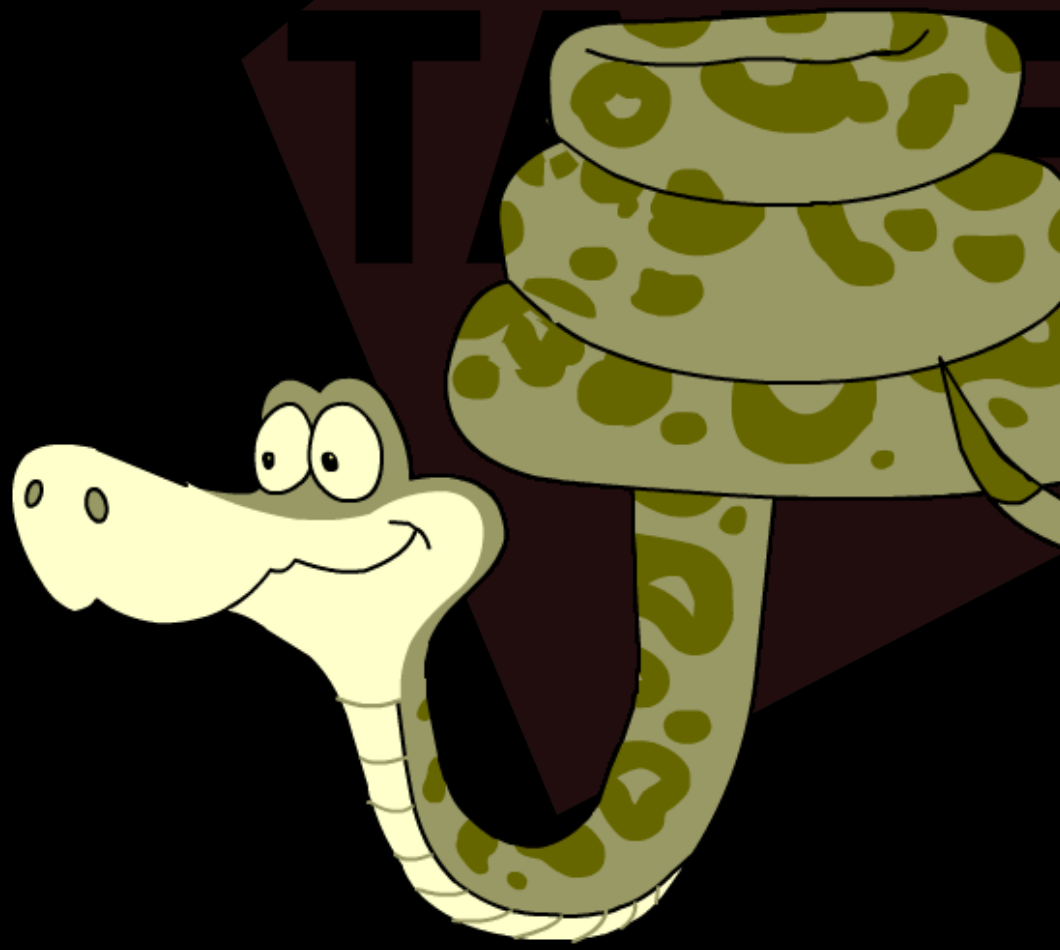
Reminder: Additional resources

Freely available online courses, that can be useful to enhance your knowledge.

- Python for Everyone (Py4e)
- LearnPython.org
- W3schools

What is a Function?

A Python function is a block of reusable code that performs a specific task.



Background created by Adrian Goold

Types of functions

In Python, there are two types of functions:

Built-in functions and **self-defined**.

Types of functions

1. **Built-in functions**

   provided by Python: print(), input(), float(), int()...

2. **Functions that we define ourselves**

   which we then use / reuse.

Why use functions?

Functions are useful because they:

- organise code
- allow readability, and;
- reduce redundancy.

Background created by Adrian Goold

More reasons to use functions

- Write code once, use multiple times
- Break down problems into manageable parts
- Make code easier to read and understand
- Test individual parts of the code independently.

NOTE: We treat function names as "new" reserved words (i.e., we avoid them as variable names)

Defining a function

A Python function uses the def reserved word.

The function is called (invoked) using:

- the function name
- parentheses ()
- arguments contained within an expression.

Note: Arguments will be explained further in the following slides.

Let's code!

Example 1: Built-in functions

Syntax:

```
name = input ("What is your name? ")
```

in-built function

Explanation:

The input function is called and does the following:

- Displays the text string received from the main program when it was called.
- Waits until user presses enter.
- User input is *returned* to the main program
- This input (data) is assigned to the *name* variable.

Example 2: Self-defined functions

Syntax:

```
def greeting():
        print("Hello, how are you?")


# main program
greeting()
```

Explanation:

- Defines function: def greeting():
- Function body: print("Hello, how...?")
- Main program: calls the greeting() function
- Call function: greeting() executes the print function.

Note: the body of the function must be indented.

Creating our own functions

New functions are created using the def keyword.

Functions must be defined before they are called.

```
def song_lyrics():
    print("Because I'm happy")
    print("Clap along if you feel like a room without a roof.")
```

Creating our own functions

This defines the function but does not execute the body of the function....yet.

```
def print_song_lyrics():
    print("Because I'm happy")
    print("Clap along if you feel like a room without a roof.")
```

Executing a function

To execute the function, you must include another line of code.

```
def print_song_lyrics(): # This line defines the function without parameters
    print("Because I'm happy")
    print("Clap along if you feel like a room without a roof.")

# main program
print_song_lyrics()  # This line calls the function without any arguments
```

Parameters and Arguments

In programming, "parameters" and "arguments" are terms you'll often hear. They are related but not the same.

Parameters: Are placeholders in a function definition. Specifies what kind of data the function expects.

Arguments: Are the actual values you pass to the function when you call it.

Parameters

A parameter is a variable inside the parentheses () in the function definition.

Syntax:

```
def greeting(name): # name is the parameter
```

Arguments

An argument is the input value given to a function when called.

- Arguments enable the function to perform different tasks each time it is called.

Syntax:

```
greeting("Sam"): # "Sam" is the argument
```

Example 1: Parameters & Arguments

```
def greeting(name): # name is the parameter
    print ("Hello, " + name + "!")

# Calling the function with an argument
greeting("Sam") # "Sam" is the argument
```

Example 1: Explained

Syntax:

```
def greeting(name):
        print("Hello, " + name + "!")


# Main program
name = input("What is your name? ")
greeting(name)
```

Explanation:

This function takes one argument, 'name', and prints a personalised greeting message.

- Define function: `def greeting(name):`
- Function body: `print("Hello, " + name + "!")`
- Main program: Prompt user for their name with `input("What is your name? ")`
- Call function: `greeting(name)` to print the greeting message.

Note: the function must be indented.

Example 2: Parameters & Arguments

```
def print_song_lyrics(song_title): # song_title is the parameter
    print("Song Title: " + song_title)
    print("Because I'm happy")
    print("Clap along if you feel like a room without a roof.")

# Calling the function with an argument
print_song_lyrics("Happy") # "Happy" is the argument
```

Void (non-fruitful) Functions

- When functions do not return a value it is a "void" function
- Functions that return values are "fruitful" functions
- Void functions are "not fruitful".

Example: Void Functions

```
def print_message():
    print("Learning functions is fun.")  # This function prints a message but does not return any value

print_message() # Calling the void function
```

In this example, print_message is a void function.
It prints a message but does not return anything.

Return Values

A return value is the result that a function gives back after completing its task.

When a function executes, it can perform calculations, process data, or perform other actions.

A function without a return value is considered a void function because it does not explicitly return anything.

Example: Fruitful Functions

```
def add(number_1, number_2):
    sum = number_1 + number_2
    return sum # This function now returns the sum

# Main program
number_1 = int(input("Type a number: "))
number_2 = int(input("Type a number: "))

answer = add(number_1, number_2) # Stores the returned value in "answer"
print("The sum is: ", answer) # Prints the returned value
```

Multiple Parameters & Arguments

- We can define multiple parameters in the function definition
- We add more arguments when we call the function
- The number and order of arguments should match the parameters.

Function Tips

- Don't repeat yourself – create reuseable code
- Break up code into logical chunks
- Make a library of common functions you could reuse – perhaps share with your friends.

Function Libraries

A function library is a collection of pre-written functions.

Function libraries are used to:

- perform tasks without re-writing code from scratch
- reuse code, improve efficiency and organisation
- provide a set of standardised, easily integrated functions.

More PEP8 Standards

Examples:

- Use lowercase letters with words separated by underscores. Example: def my_function():
- Use 4 spaces per indentation level
- Limit lines to 79 characters
- Use two blank lines to separate function definitions from other code.

Summary

- Creating and calling functions
- Function parameters
- Function arguments
- Return values
- More PEP8 standards

Activity to complete

Practise what you have learnt today!

Locate your Week 5 Revision exercises folder and complete the following activity:

1. Build a simple calculator.